Livre dont vous êtes le héros

TP de programmation Python – un jeu en pages

```
> python moteur.py histoire_A12_
```

1 Le principe

Dans un *livre dont vous êtes le héros*, le lecteur ne lit pas les pages dans l'ordre : à la fin de chaque paragraphe, il fait un choix (« Pour ouvrir la porte, allez page 12 ; pour fuir, allez page 7 »). Vous allez écrire un jeu qui fonctionne de la même façon.

Le thème est **libre** : aventure, enquête policière, escape game, exploration spatiale, histoire interactive… Tout jeu convient, du moment qu'il respecte le fonctionnement décrit ci-dessous.

- Chaque **page** de votre histoire est une **fonction Python** écrite dans son propre fichier.
- Une page affiche du texte, pose éventuellement une question au joueur, puis **retourne le numéro de la page suivante**.
- Les informations du jeu (par exemple nom du joueur, points de vie, inventaire, indices trouvés, score…) sont rangées dans un **dictionnaire** appelé `gameState`, qui est transmis d'une page à l'autre.
- Toutes vos fonctions sont **typées** : on précise le type de chaque paramètre et celui de la valeur retournée.
- Avant la page 1, une fonction `menu()` **prépare la partie** et crée le dictionnaire `gameState` : création d'un personnage, choix d'un niveau de difficulté, présentation des règles… selon ce dont votre jeu a besoin.

Votre travail consiste **uniquement** à écrire la fonction `menu()`, les fonctions `pageX(gameState)` et, si vous le souhaitez, des dessins en ASCII art et des fonctions annexes (combats, énigmes…).

2 Le moteur

Le fichier `moteur.py` vous est fourni. Son rôle est simplement d'**enchaîner vos pages** : il appelle `menu()`, puis `page1`, puis la page dont le numéro a été retourné, et ainsi de suite jusqu'à la fin du jeu. Il vérifie aussi votre code et affiche un message d'erreur explicite quand quelque chose ne va pas (page manquante, cas non prévu, mauvais type de retour, etc.). Lisez ces messages : ils vous disent précisément quoi corriger.

△ Vous ne devez pas modifier le moteur

Vous n'avez **pas besoin** de comprendre le code de `moteur.py` et vous **ne devez pas le modifier**, ni `ASCII/ascii_gameover.py`. Votre projet sera testé avec la version d'origine de ces fichiers : si votre jeu ne fonctionne qu'avec un moteur modifié, il ne fonctionnera pas le jour de l'évaluation.

3 Organisation de l'archive

Voici le contenu de l'archive fournie :

```
projet/
├─ moteur.py  ← ne pas modifier
├─ promptAcopierColler.txt  le prompt pour l'IA (partie 10)
├─ ASCII/  les dessins, un fichier par dessin
│  ├─ ascii_gameover.py  ← ne pas modifier
│  └─ ascii_dragon.py  exemple de dessin
├─ OUTILS/  vos fonctions annexes (combats...)
│  └─ combat.py  exemple, à compléter
└─ histoire_A12/  votre histoire (à renommer)
   ├─ menu.py  contient menu()
   ├─ page1.py  contient page1(gameState)
   ├─ page2.py  contient page2(gameState)
   └─ ...
```

- Le dossier `histoire_A12` est un exemple : renommez-le `histoire_` suivi de l'identifiant de votre équipe donné par votre enseignant (groupe de TD + numéro).
- Pour lancer le jeu, ouvrez un terminal **dans le dossier `projet`** et tapez :

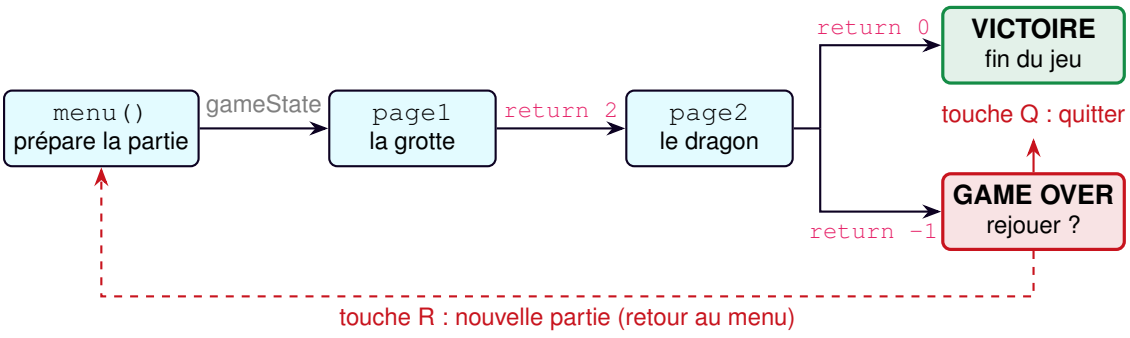
```
python moteur.py histoire_A12
```

Le terminal à utiliser

L'écran de GAME OVER est en couleurs : lancez le jeu depuis le terminal de VS Code, le terminal Windows, ou celui de Linux / macOS. Dans IDLE ou Thonny, le jeu fonctionne mais le GAME OVER s'affiche en version simplifiée, sans couleurs.

4 L'enchaînement des pages

Le schéma ci-dessous montre le déroulement de l'exemple fourni. Chaque flèche correspond à un `return` : c'est la valeur retournée par une page qui décide de la suite.



Trois valeurs de retour sont spéciales :

- `0` : le jeu se termine normalement (victoire, fin de l'histoire) ;
- `-1` : le joueur a perdu, l'écran GAME OVER s'affiche et propose de recommencer une partie (retour à `menu()`) ou de quitter ;
- `-2` : **cas non prévu**. C'est la valeur par défaut de vos pages (voir la convention `res` en partie 9). Si une page retourne `-2`, c'est que vous avez oublié un cas : le moteur s'arrête et vous indique la page en cause.

5 Le dictionnaire `gameState`

`gameState` contient tout ce dont votre jeu a besoin de se souvenir. C'est vous qui choisissez ses clés dans `menu()`. Dans l'exemple fourni, `menu()` crée un personnage ; dans votre jeu, elle peut faire tout autre chose. Les pages le lisent et le modifient directement : les modifications sont conservées pour les pages suivantes, il n'y a **pas besoin de le retourner**.

```python
# menu.py
def menu() -> dict:
    print("=== CRÉATION DU PERSONNAGE ===")
    nom: str = input("Quel est votre nom ? ")
    gameState: dict = {
        "nom": nom,
        "force": 7,
        "vie": 12,
        "inventaire": [],
    }
    return gameState      # menu() est la seule fonction qui retourne le
    dictionnaire
```

6 Ajouter une page

Pour ajouter la page 3 :

1. créez le fichier `page3.py` dans votre dossier d'histoire ;
2. écrivez dedans une fonction nommée **exactement** `page3`, qui prend un seul paramètre, `gameState`, et qui est **typée** : `def page3(gameState: dict) -> int:` ;
3. première ligne de la fonction : `res = -2` ; dernière ligne : `return res` (c'est la convention `res`, voir partie 9) ;
4. entre les deux, faites en sorte que **chaque cas** donne à `res` le numéro de la page suivante (ou `0` / `-1`) ;
5. vérifiez qu'une autre page retourne bien `3`, sinon votre page ne sera jamais atteinte.

```python
# page3.py
def page3(gameState: dict) -> int:
    res: int = -2              # par défaut : cas non prévu

    print("Deux couloirs s'ouvrent devant vous.")
    print("1 - Le couloir de gauche, d'où vient une odeur de soufre")
    print("2 - Le couloir de droite, plongé dans le noir")
    choix: str = input("> ")

    if choix == "1":
        res = 5
    elif choix == "2":
        gameState["vie"] = gameState["vie"] - 2
        print("Vous trébuchez dans le noir et perdez 2 points de vie.")
        if gameState["vie"] <= 0:
            res = -1           # plus de vie : GAME OVER
        else:
            res = 4
    else:
        print("Choix invalide.")
        res = 3                # on reste sur la même page

    return res
```

Si vous oubliez le `else` final et que le joueur tape autre chose que 1 ou 2, `res` garde la valeur `-2` et le moteur vous le signale :

```
ERREUR : La page 3 a retourné -2 : CAS NON PRÉVU.
```

Les erreurs les plus fréquentes

- Le fichier s'appelle `page3.py` mais la fonction s'appelle `page_3` ou `Page3` : les deux noms doivent être identiques.
- Un cas oublié (souvent le choix invalide) : `res` reste à `-2`.
- Le typage est oublié ou incomplet : le moteur refuse de lancer la page et vous rappelle la signature attendue.
- La page retourne `"4"` (du texte) au lieu de `4` (un nombre). Si le numéro vient d'un `input()`, convertissez-le avec `int(...)`.
- Une page appelle directement une autre page (`page4(gameState)`) au lieu de donner son numéro à `res` (`res = 4`).

7 Ajouter un dessin en ASCII art

Les dessins sont rangés dans le dossier `ASCII`, un dessin par fichier. Le fichier `ascii_dragon.py` sert de modèle.

1. Copiez `ascii_dragon.py` dans le dossier `ASCII` sous un nouveau nom commençant par `ascii_`, par exemple `ascii_tresor.py`.
2. Renommez la fonction pour qu'elle porte **le même nom que le fichier** et gardez le typage : `def ascii_tresor() -> None:`.

3. Collez votre dessin entre les triple guillemets du `print`. **Gardez le `r` devant les guillemets** : sans lui, Python interprète mal les barres obliques inversées (`\`) du dessin.
4. Testez le dessin seul : `python ASCII/ascii_tresor.py`.

```
# ASCII/ascii_tresor.py
def ascii_tresor() -> None:
    print(r"""
      _________
     /        /|
    /________/ |
    |  $$$   | |
    |________|/
""")
```

Pour l'afficher dans une page, importez la fonction en haut du fichier de la page, puis appelez-la :

```
# page5.py
from ASCII.ascii_tresor import ascii_tresor


def page5(gameState: dict) -> int:
    res: int = -2

    ascii_tresor()
    print("Vous découvrez un coffre rempli de pièces d'or !")
    gameState["inventaire"].append("or")
    res = 6

    return res
```

8 Écrire des fonctions annexes

Certaines actions reviennent souvent dans une histoire : un combat, un marchand, un lancer de dé… Plutôt que de recopier le même code dans plusieurs pages, écrivez-le une seule fois dans une **fonction annexe**, rangée dans le dossier `OUTILS`. Contrairement aux pages, un fichier du dossier `OUTILS` peut contenir **plusieurs fonctions**.

Le fichier `OUTILS/combat.py` vous est fourni comme exemple. Il contient :

- une fonction générique `combat(gameState, monstre)`, où `monstre` est un dictionnaire (`"nom"`, `"vie"`, `"force"`). Elle retourne `True` (type `bool`) si le joueur gagne, `False` sinon. **Son contenu est à écrire par vous** ;
- des fonctions spécialisées, `combat_dragon(gameState)` et `combat_orc(gameState)`, qui créent chacune leur monstre puis appellent `combat`.

```python
# OUTILS/combat.py
def combat(gameState: dict, monstre: dict) -> bool:
    res: bool = False

    # A IMPLEMENTER

    return res


def combat_dragon(gameState: dict) -> bool:
    monstre: dict = {"nom": "dragon", "vie": 20, "force": 4}
    res: bool = combat(gameState, monstre)
    return res
```

Pour ajouter un monstre, écrivez une nouvelle fonction sur le modèle de `combat_dragon`.

Dans une page, importez la fonction puis utilisez son résultat pour choisir la suite :

```python
# page2.py
from ASCII.ascii_dragon import ascii_dragon
from OUTILS.combat import combat_dragon


def page2(gameState: dict) -> int:
    res: int = -2

    ascii_dragon()
    print("Un dragon surgit et vous barre la route !")
    if combat_dragon(gameState):
        res = 0      # victoire
    else:
        res = -1     # GAME OVER

    return res
```

△ Seules les pages choisissent la page suivante

Une fonction annexe retourne une **information** (gagné ou perdu, un objet trouvé, une somme d'argent...), **jamais un numéro de page**. C'est toujours la page qui décide de la suite, à partir de cette information.

9 Consignes à respecter

CONSIGNES OBLIGATOIRES

Un projet qui ne respecte pas ces consignes **ne fonctionnera pas avec le moteur** et sera pénalisé.

1. **Ne modifiez jamais** `moteur.py` ni `ASCII/ascii_gameover.py`.
2. Ne renommez pas et ne déplacez pas les fichiers fournis. Seul le dossier `histoire_A12` doit être renommé avec l'identifiant de votre équipe.
3. **Un fichier par page.** Le fichier `pageX.py` contient **une seule fonction**, nommée **exactement** `pageX`, avec **un seul paramètre** : `gameState`.
4. `menu.py` contient la fonction `menu()`, **sans paramètre**, qui **retourne un dictionnaire**.
5. Chaque page **retourne toujours un nombre entier** : le numéro d'une page qui existe, `0` pour la fin du jeu, `-1` pour le GAME OVER. **Appliquez les conventions** décrites dans l'encadré qui suit (un seul `return`, typage, pas de `break`).
6. Une page **n'appelle jamais** une autre page : elle retourne son numéro.
7. **Toutes vos fonctions sont typées**, paramètres et valeur de retour comprises : `def menu() -> dict:`, `def page4(gameState: dict) -> int:`. Le moteur refuse une fonction non typée.
8. Toutes les informations du jeu sont stockées dans `gameState`. **Aucune variable en dehors des fonctions**, pas de variables globales.
9. Un fichier de page ne contient **rien d'autre** que sa fonction et, éventuellement, les `import` de vos dessins (`ASCII`) et de vos fonctions annexes (`OUTILS`).
10. Les fonctions annexes sont rangées dans `OUTILS` et ne retournent **jamais** un numéro de page.
11. Chaque dessin est dans `ASCII/ascii_nom.py` et contient une fonction `ascii_nom()`, sans paramètre.
12. **Avant de rendre votre travail**, lancez `python moteur.py histoire_XXX` et parcourez **tous les chemins** de votre histoire, y compris les choix invalides. Le jeu ne doit afficher aucun message d'erreur.

CONVENTIONS : `res`, TYPAGE, PAS DE `break`

1. Un seul `return` par fonction (la variable `res`)

Toutes vos fonctions qui retournent une valeur s'écrivent sur le même modèle :

- **première ligne** : créez la variable `res` (pour « résultat ») et donnez-lui une **valeur par défaut** ;
- **dernière ligne** : `return res`, et c'est le **seul** `return` de la fonction ;
- **entre les deux** : assurez-vous que, dans chaque cas, `res` reçoit la bonne valeur.

```
def page7(gameState: dict) -> int
    :
    res: int = -2

    # chaque cas : res = ...

    return res
```

Valeurs par défaut :

- pages : **−2** (cas non prévu, le moteur affiche une erreur) ;
- fonctions annexes : une valeur adaptée à ce qu'elles retournent (`False`, `0`, `""`...).

2. Tout est typé

Indiquez le type de chaque paramètre et celui de la valeur retournée, ainsi que le type des variables importantes. Les types utiles ici : `int`, `float`, `str`, `bool`, `list`, `tuple`, `dict`, et `None` pour une fonction qui ne retourne rien (un dessin ASCII, par exemple).

```
def page7(gameState: dict) -> int:
    res: int = -2
    choix: str = input("> ")
    ...
    return res
```

3. Une boucle s'arrête uniquement par sa condition

`break`, `continue` et `while True` sont interdits. La condition écrite en haut du `while` doit dire, à elle seule, quand la boucle s'arrête. Par exemple, pour redemander un choix tant qu'il n'est pas valide :

```
choix: str = input("> ")
while choix != "1" and choix != "2":
    print("Choix invalide.")
    choix = input("> ")
```

Pourquoi ces conventions ?

Ce sont des **choix pédagogiques**, pas des règles universelles de la programmation : un développeur expérimenté utilise parfois `break` ou plusieurs `return`, en connaissance de cause, et Python n'oblige pas à typer. **Dans ce TP, elles sont en revanche obligatoires.** Les respecter coûte en général une ou deux lignes de code, mais vous oblige à réfléchir à la logique de votre programme avant de l'écrire : c'est exactement ce que nous cherchons à vous faire acquérir.

9.1 Notions interdites

Ce TP est un exercice d'initiation. Vous devez utiliser **uniquement les notions vues en cours** : variables, types de base, `input` / `print`, conditions (`if` / `elif` / `else`), boucles (`for`, `while`), fonctions, listes, tuples et dictionnaires. En particulier, les éléments suivants sont **interdits** :

- les classes et la programmation orientée objet (`class`, `self`) ;
- les variables globales (`global`, `nonlocal`) ;
- les fonctions `lambda` ;
- les listes ou dictionnaires en compréhension (`[x for x in ...]`) ;
- les exceptions (`try` / `except`, `raise`) ;
- l'instruction `match` / `case` ;
- `break`, `continue` et `while True` (voir les conventions) ;
- les décorateurs (`@...`) et les générateurs (`yield`) ;
- l'opérateur « morse » `:=` ;
- `*args`, `**kwargs` ;
- la lecture et l'écriture de fichiers (`open`, `with`) ;
- `exit()`, `quit()`, `sys.exit()` : ils arrêtent le moteur ;
- les bibliothèques externes (installées avec `pip`) ;
- tout `import` autre que vos fichiers `ASCII` et `OUTILS`, sauf accord de votre enseignant.

En cas de doute sur une notion, **demandez à votre enseignant avant de l'utiliser**. Vous devez être capables d'expliquer chaque ligne de votre code.

10 Utiliser l'IA intelligemment

Vous avez le droit d'utiliser une IA (ChatGPT, Gemini, Claude, Le Chat…), mais comme un **professeur particulier**, pas comme quelqu'un qui fait le travail à votre place. Une IA à qui on demande « écris-moi la page 3 » produira souvent du code qui utilise des notions interdites, que vous ne saurez pas expliquer, et qui ne respecte pas les consignes du moteur.

Utilisez-la plutôt pour :

- comprendre une notion du cours (« comment fonctionne un dictionnaire ? ») ;
- comprendre un message d'erreur ;
- vérifier votre raisonnement sur un petit exemple ;
- obtenir des exercices pour vous entraîner.

Pour que l'IA se comporte comme un tuteur adapté à ce TP, donnez-lui les instructions ci-dessous. Vous pouvez les coller au début de chaque conversation, ou mieux, les enregistrer une fois pour toutes : dans un **Gem** (Gemini), un **GPT personnalisé** ou un **Projet** (ChatGPT), un **Projet** ou une **Skill** (Claude), ou dans les **instructions personnalisées** de votre outil.

Ce prompt est aussi fourni dans le fichier `promptAcopierColler.txt`, à la racine de l'archive : vous pouvez copier son contenu, ou simplement téléverser le fichier dans votre outil.

Prompt à copier-coller

Tu es un tuteur de programmation Python pour un étudiant en initiation à la programmation. Je réalise un TP : un jeu de type « livre dont vous êtes le héros » où chaque page est une fonction pageX(gameState) placée dans son propre fichier pageX.py, qui retourne le numéro de la page suivante (0 pour la fin du jeu, -1 pour le game over). gameState est un dictionnaire qui contient l'état du jeu. Un moteur fourni par l'enseignant enchaîne les pages : je ne dois pas le modifier. Je peux aussi écrire des fonctions annexes (par exemple des combats) qui retournent une information, jamais un numéro de page.

Conventions imposées : toutes les fonctions sont typées (par exemple def page3(gameState: dict) -> int:) ; chaque fonction commence par res = valeur par défaut (-2 pour une page, qui signifie « cas non prévu ») et se termine par un unique return res.

Règles que tu dois respecter :

1. Ne m'écris jamais le code de mes pages ni la solution complète d'un exercice. Aide-moi à trouver par moi-même : explique la notion, pose-moi des questions, donne-moi des indices progressifs.
2. Si tu donnes un exemple de code, il doit être court (moins de 10 lignes) et porter sur un autre sujet que mon jeu.
3. Dans tous tes exemples, respecte ces conventions : fonctions et variables typées ; un seul return par fonction, en dernière ligne (variable res) ; jamais de break, continue ou while True, une boucle s'arrête uniquement par sa condition.
4. N'utilise que les notions suivantes : variables, types de base, input, print, if / elif / else, boucles for et while, fonctions, listes, tuples, dictionnaires et annotations de type simples (int, str, bool, float, list, tuple, dict, None). N'utilise jamais : classes, global, lambda, compréhensions de listes, try / except, match / case, yield, décorateurs, :=, *args / **kwargs, exit / quit / sys.exit, fichiers, bibliothèques externes. Si ma question nécessite une de ces notions, dis-le-moi et propose une solution sans elle.
5. Quand je te montre un message d'erreur, explique-moi ce qu'il signifie et aide-moi à trouver la ligne en cause, sans corriger le code à ma place.
6. Explique simplement, avec des mots adaptés à un débutant, et vérifie que j'ai compris en me posant une question à la fin de tes explications.
7. Si je te demande de faire mon travail à ma place, refuse gentiment et rappelle-moi ces règles.

△ Rappel

Le code rendu doit être **le vôtre**. Vous pourrez être interrogés sur n'importe quelle partie de votre projet : un code que vous ne savez pas expliquer sera considéré comme non réalisé.